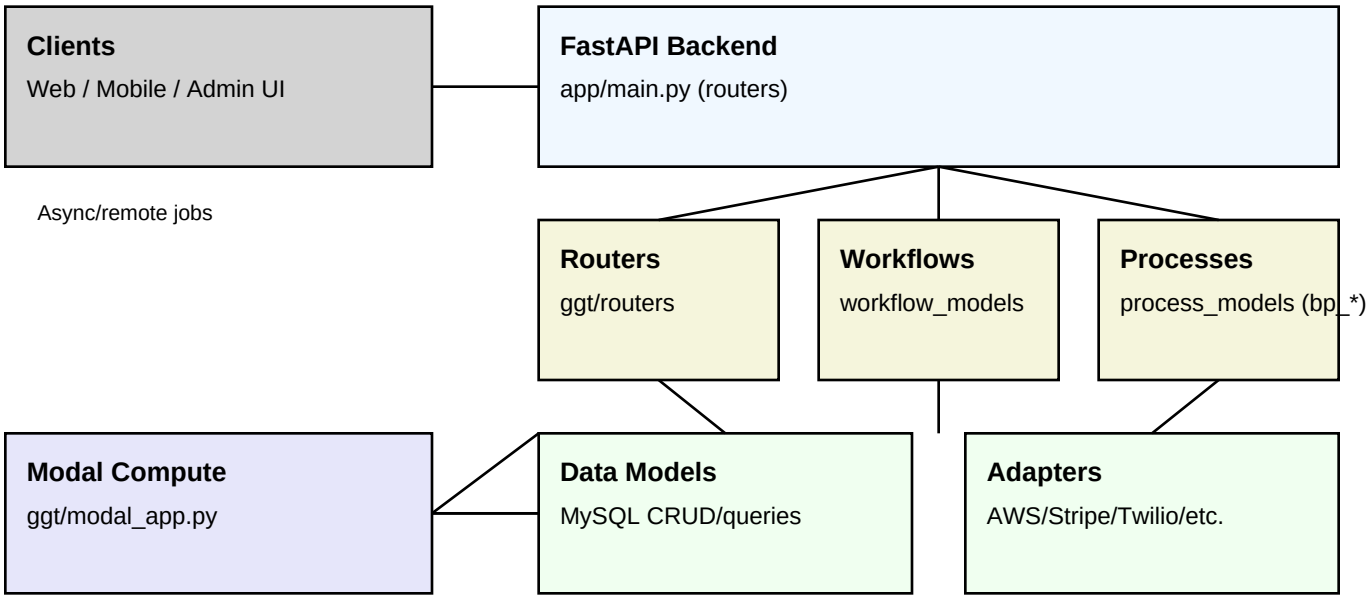
GGT Project Explanation

Generated: 2026-06-26 08:09 UTC

1. Overview

This repository is a Python FastAPI backend that exposes multiple role-based APIs (patient, portal/admin, providers, contact center, billing, reporting, vendors). It integrates with MySQL and several external services (AWS, Auth0, Twilio, SendGrid, Stripe, GCP Storage). Heavy or long-running compute can be offloaded to Modal using a dedicated Modal app.

2. Architecture Visualization



3. Frontend / Client Layer

This repo primarily contains the backend service. Frontend clients are assumed to be separate applications (web/mobile/admin UI) that call the REST endpoints. The backend organizes routes by role and prefixes (e.g., /api for patient, /api/portal for admin).

4. Backend Details

4.1 Entry points

FastAPI app wiring lives in app/main.py; routers are included from app/ggt/routers. AWS Lambda is supported via Mangum, and Modal compute is provided via app/ggt/modal_app.py.

4.2 Layering

Routers → workflow_models → process_models (bp_*) → data_models. Adapters in ggt/lib/adapters isolate external dependencies such as AWS services and vendor APIs.

5. Web Tech Stack & Technologies

Layer	Technology / Notes
API	Python + FastAPI (ASGI), Uvicorn; routers under app/ggt/routers
Business logic	workflow_models → process_models (bp_*) → data_models
Database	MySQL (mysql-connector-python)
Auth	Auth0 JWT validation (jose/jwt) + optional API key header for vendors
Messaging/Tasks	In-process BackgroundTasks plus Modal compute offload
Cloud	AWS (Secrets Manager, S3, SNS/SQS/Pinpoint, DynamoDB), GCP Storage
Payments	Stripe
Comms	Twilio (SMS/voice), SendGrid (email templates)
Docs/Run	Docker images for ASGI and Lambda; runbook under docs/runbook.md

6. Deployment & Runtime Options

The service can run as a normal ASGI web service (local Uvicorn, Docker gunicorn/uvicorn image) or as an AWS Lambda container. Modal compute can run background job functions and expose an HTTP endpoint for triggering those jobs remotely.

Operational endpoints: /healthz and /readyz. /readyz can optionally perform a MySQL connectivity check when enabled via environment variable.

7. Modal Integration (Compute Backend)

Modal compute is integrated through a Modal app that wraps selected heavy workloads (queue processors, lab integration, schedule generation). The API service can trigger Modal either by calling the deployed Modal HTTP endpoint (when MODAL_WEB_BASE_URL is set) or by SDK lookup.

Key environment variables: MODAL_APP_NAME, MODAL_SECRET_NAME, MODAL_WEB_BASE_URL, READYZ_CHECK_DB, STRICT_CONFIG.

8. Quick Run Commands (Reference)

For step-by-step commands and troubleshooting, see docs/runbook.md in the repository. Typical local dev run: cd app && ./run.sh. Docker run: docker build -t ggt-api . && docker run --rm -p 8000:80 ggt-api.